

2ª Avaliação de JCRBDD2 – Banco de Dados 2

Tema: Desenvolvimento de uma Plataforma de Engenharia de Dados para Detecção de Fraudes em Cartão de Crédito

Descrição Geral:

O grupo (de até 4 alunos) deverá projetar e implementar um pequeno projeto de engenharia de dados para detecção de fraude em cartão crédito. O projeto contará com um *pipeline* de dados automatizado com um ETL que fará a ingestão de dados a partir de um Banco de Dados Relacional MySQL e um Banco de Dados NoSQL Mongo, criando em seguida um *Data Lakehouse* simulado (contendo arquivos txt, *dataframe* e tabelas relacionais). Os dados no *pipeline* sofrerão as atualizações e transformações necessárias para serem entregues adequadamente aos analistas de Ciência de Dados.

1) Requisitos Técnicos Obrigatórios:

1. Máquina Virtual

- Utilizar máquina virtual Ubuntu Server versão 25, disponibilizada no Moodle com *docker*, *python3*, ambiente *.venv*, *Apache Airflow* e outros aplicativos já instalados.

2. Bancos de Dados

- SGBD MySQL executado em container *docker*.
- MongoDB acessado via nuvem (MongoDB Atlas).
- Utilizar como massa de dados o arquivo *credit-card.csv*, que deverá ser particionado em dois arquivos de origem:
 1. *credit-card1.csv*, com a primeira metade dos dados do arquivo *credit-card.csv* original, e que deverá ser ingerido no MySQL; e
 2. *credit-card2.csv*, com a segunda metade dos dados do arquivo *credit-card.csv* original que deverá ser transformado no arquivo *credit-card2.json* e ingerido no MongoDB.
- Arquitetura Medalhão (*Medallion Architecture*) para armazenamento no *Data Lakehouse* simulado.
 1. Na Camada *Bronze* armazenar dados no formato txt.
 2. Na Camada *Silver* armazenar dados (dados com as *features* relevantes e agregações ou sumarizações) no formato *pandas dataframe*
 3. Na Camada *Gold* armazenar os dados em formato de tabelas relacionais, usando o *Sqlite*.

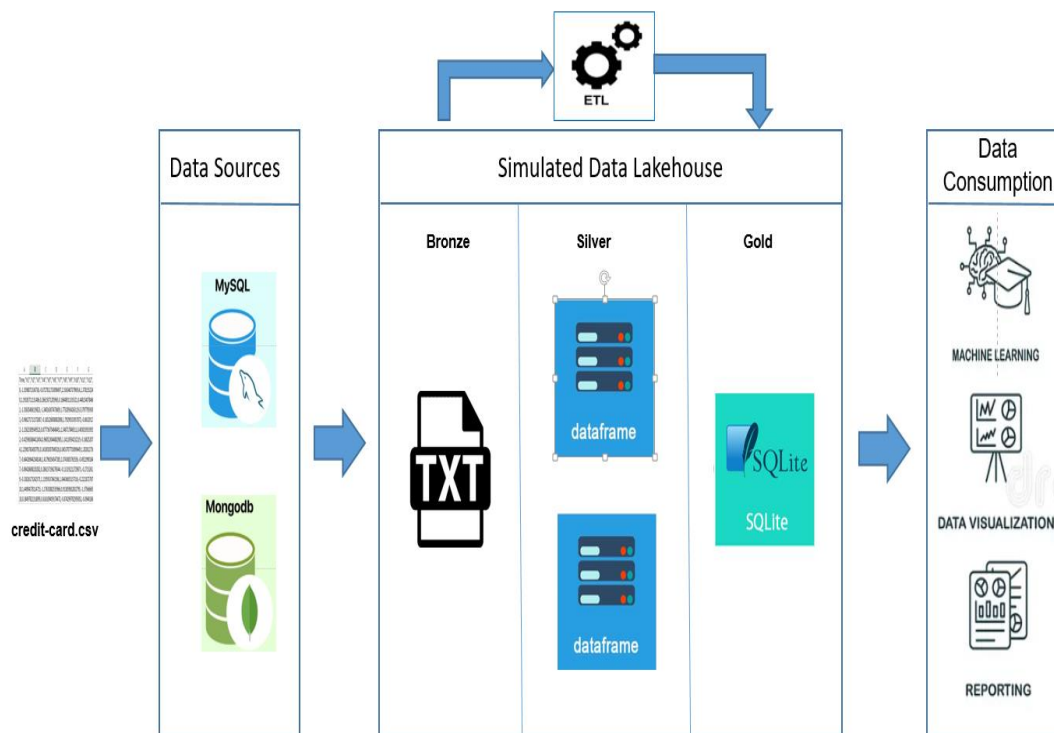
3. ETL

- Utilizar *python* para implementação do ETL.
- Automatizar o ETL utilizando a ferramenta *Apache AirFlow*.

4. Data Science

- Fica a critério dos desenvolvedores utilizar o algoritmo de Aprendizagem de Máquina (*Machine Learning*) mais conveniente para a classificação dos dados de cartão crédito.

2) Arquitetura do Projeto:



3) Entregáveis:

- **Relatório Técnico** explicando o que foi realizado.
- **Código-fonte Funcional** com README explicando como executar o *pipeline*.
- **Apresentação em Grupo**, com no máximo 10 minutos, mostrando casos de uso executados em tempo real.

4) Critérios de avaliação:

- **Qualidade da Implementação** (boas práticas de programação).
- **Complexidade Resolvida** (automação, monitoramento, etc.).
- **Trabalho em Grupo** (divisão clara das partes envolvidas: implementação, documentação, ou seja, a definição de atuação de cada membro do grupo).
- **Demonstração Prática** (funcionamento do sistema ao vivo).

5) Referências:

a) Máquina Virtual Ubuntu Server 25.10 – arquivo JCRBDD2-2aProva.ova:
<https://drive.google.com/drive/folders/1pFsZRfeD7vrXcnmYjHu5jnMkK7Ya v3vu?usp=sharing>

- b) Dados de cartão de crédito (fonte do arquivo *credit-card.csv*):
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.
- c) *Notebooks* de avaliação (escolher o que achar mais conveniente para se basear na detecção de fraude em cartão de crédito):
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/code>
- d) MySQL + Docker - Subindo um banco de dados MySQL em um container Docker (*docker* já está instalado na máquina virtual):
<https://johnfercher.medium.com/mysql-docker-7ff6d50d6cf1>
- e) How To Install and Use SQLite on Ubuntu 20.04:
<https://www.digitalocean.com/community/tutorials/how-to-install-and-use-sqlite-on-ubuntu-20-04>
- f) How to Use Python with MongoDB (using Mongo Atlas):
<https://www.mongodb.com/resources/languages/python>